
HOY
Clase Gratuita
Google -Meet
17
JULIO
20:00 PM
21:00 PM
19:00 PM
22:00 PM
Aprende
PYTHON
Y Logra una Certificacion
Internacional
Gratis - Se Entregara Materiales del curso a los Participantes - Gratis

Strings

In [ ]:
```
{'abcde':10}                            # 'abcde     '
{'abcde':10.3}                          # 'abc       '
{'abcde':.3}                            # 'abc'
{'abcde'!r:10}                          # "'abcde'   "
```

Numbers

In [ ]:
```
{123456:10}                             # '    123456'
{123456:10,}                            # '   123,456'
{123456:10_}                            # '   123_456'
{123456:+10}                            # '   +123456'
{123456:=+10}                           # '+   123456'
{123456: }                              # ' 123456'
{-123456: }                             # '-123456'
<int>      = int(<float/str/bool>)                # Or: math.floor(<float>)
<float>    = float(<int/str/bool>)                # Or: <real>e±<int>
<complex>  = complex(real=0, imag=0)              # Or: <real> ± <real>j
<Fraction> = fractions.Fraction(0, 1)             # Or: Fraction(numerator=0, denominator=1)
<Decimal>  = decimal.Decimal(<str/int>)           # Or: Decimal((sign, digits, exponent))
```

Floats

In [ ]:
```
{1.23456:10.3}                          # '      1.23'
{1.23456:10.3f}                         # '     1.235'
{1.23456:10.3e}                         # ' 1.235e+00'
{1.23456:10.3%}                         # '  123.456%'
```

Comparison of presentation types:

In [ ]:
```
+--------------+----------------+----------------+----------------+----------------+
|              |    {<float>}   |   {<float>:f}  |   {<float>:e}  |   {<float>:%}  |
+--------------+----------------+----------------+----------------+----------------+
|  0.000056789 |   '5.6789e-05' |    '0.000057'  | '5.678900e-05' |    '0.005679%' |
|  0.00056789  |   '0.00056789' |    '0.000568'  | '5.678900e-04' |    '0.056789%' |
|  0.0056789   |   '0.0056789'  |    '0.005679'  | '5.678900e-03' |    '0.567890%' |
|  0.056789    |   '0.056789'   |    '0.056789'  | '5.678900e-02' |    '5.678900%' |
|  0.56789     |   '0.56789'    |    '0.567890'  | '5.678900e-01' |   '56.789000%' |
|  5.6789      |   '5.6789'     |    '5.678900'  | '5.678900e+00' |  '567.890000%' |
| 56.789       |  '56.789'      |   '56.789000'  | '5.678900e+01' | '5678.900000%' |
+--------------+----------------+----------------+----------------+----------------+
```

In [ ]:
```
+--------------+----------------+----------------+----------------+----------------+
|              |  {<float>:.2}  |  {<float>:.2f} |  {<float>:.2e} |  {<float>:.2%} |
+--------------+----------------+----------------+----------------+----------------+
|  0.000056789 |    '5.7e-05'   |      '0.00'    |   '5.68e-05'   |      '0.01%'   |
|  0.00056789  |    '0.00057'   |      '0.00'    |   '5.68e-04'   |      '0.06%'   |
|  0.0056789   |    '0.0057'    |      '0.01'    |   '5.68e-03'   |      '0.57%'   |
|  0.056789    |    '0.057'     |      '0.06'    |   '5.68e-02'   |      '5.68%'   |
|  0.56789     |    '0.57'      |      '0.57'    |   '5.68e-01'   |     '56.79%'   |
|  5.6789      |    '5.7'       |      '5.68'    |   '5.68e+00'   |    '567.89%'   |
| 56.789       |    '5.7e+01'   |     '56.79'    |   '5.68e+01'   |   '5678.90%'   |
+--------------+----------------+----------------+----------------+----------------+
```

Bin, Hex

In [ ]:
```
<int> = ±0b<bin>                        # Or: ±0x<hex>
<int> = int('±<bin>', 2)                # Or: int('±<hex>', 16)
<int> = int('±0b<bin>', 0)              # Or: int('±0x<hex>', 0)
<str> = bin(<int>)                      # Returns '[-]0b<bin>'.
```

Bitwise Operators

In [ ]:
```
<int> = <int> & <int>                          # And (0b1100 & 0b1010 == 0b1000).
<int> = <int> | <int>                          # Or  (0b1100 | 0b1010 == 0b1110).
<int> = <int> ^ <int>                          # Xor (0b1100 ^ 0b1010 == 0b0110).
<int> = <int> << n_bits                        # Left shift. Use >> for right.
<int> = ~<int>                                 # Not. Also -<int> - 1.
```

Colleciones

Manejo de Listas

In [ ]:
```
<list>.append(<el>)            # Or: <list> += [<el>]
<list>.extend(<collection>)    # Or: <list> += <collection>
<list>.sort()                  # Sorts in ascending order.
<list>.reverse()               # Reverses the list in-place.
<list> = sorted(<collection>)  # Returns a new sorted list.
<iter> = reversed(<list>)      # Returns reversed iterator.
sum_of_elements  = sum(<collection>)
elementwise_sum  = [sum(pair) for pair in zip(list_a, list_b)]
sorted_by_second = sorted(<collection>, key=lambda el: el[1])
sorted_by_both   = sorted(<collection>, key=lambda el: (el[1], el[0]))
flatter_list     = list(itertools.chain.from_iterable(<list>))
product_of_elems = functools.reduce(lambda out, el: out * el, <collection>)
list_of_chars    = list(<str>)
<list>.insert(<int>, <el>)     # Inserts item at index and moves the rest to the right.
<el>  = <list>.pop([<int>])    # Removes and returns item at index or from the end.
<int> = <list>.count(<el>)     # Returns number of occurrences. Also works on strings.
<int> = <list>.index(<el>)     # Returns index of the first occurrence or raises ValueError.
<list>.remove(<el>)            # Removes first occurrence of the item or raises ValueError.
<list>.clear()                 # Removes all items. Also works on dictionary and set.
```

Manejo de Diccionarios

In [ ]:
```
<view> = <dict>.keys()  # Coll. of keys that reflects changes.
<view> = <dict>.values()# Coll. of values that reflects changes.
<view> = <dict>.items()# Coll. of key-value tuples that reflects chgs.
value  = <dict>.get(key, default=None)          # Returns default if key is missing.
value  = <dict>.setdefault(key, default=None)   # Returns and writes default if key is missing.
<dict> = collections.defaultdict(<type>)        # Returns a dict with default value of type.
<dict> = collections.defaultdict(lambda: 1)     # Returns a dict with default value 1.
<dict> = dict(<collection>)                     # Creates a dict from coll. of key-value pairs.
<dict> = dict(zip(keys, values))                # Creates a dict from two collections.
<dict> = dict.fromkeys(keys [, value])          # Creates a dict from collection of keys.
<dict>.update(<dict>)                           # Adds items. Replaces ones with matching keys.
value = <dict>.pop(key)                         # Removes item or raises KeyError.
{k for k, v in <dict>.items() if v == value}    # Returns set of keys that point to the value.
{k: v for k, v in <dict>.items() if k in keys}  # Returns a dictionary, filtered by keys.
```

Manejo de Sets

In [ ]:
```
<set> = set()  # `{}` returns a set.
<set>.add(<el>)                    # Or: <set> |= {<el>}
<set>.update(<collection> [, ...]) # Or: <set> |= <set>
<set>  = <set>.union(<coll.>)                   # Or: <set> | <set>
<set>  = <set>.intersection(<coll.>)            # Or: <set> & <set>
<set>  = <set>.difference(<coll.>)              # Or: <set> - <set>
<set>  = <set>.symmetric_difference(<coll.>)    # Or: <set> ^ <set>
<bool> = <set>.issubset(<coll.>)                # Or: <set> <= <set>
<bool> = <set>.issuperset(<coll.>)              # Or: <set> >= <set>
<el> = <set>.pop()                              # Raises KeyError if empty.
<set>.remove(<el>)                              # Raises KeyError if missing.
<set>.discard(<el>)                             # Doesn't raise an error.
```

Manejo de Tuplas

```
<tuple> = ()                                # Empty tuple.
```

```
<tuple> = (<el>,)                       # Or: <el>,
<tuple> = (<el_1>, <el_2> [, ...])      # Or: <el_1>, <el_2> [, ...]
```

Comprensiones

In [ ]:
```
<list> = [i+1 for i in range(10)]                   # Or: [1, 2, ..., 10]
<iter> = (i for i in range(10) if i > 5)            # Or: iter([6, 7, 8, 9])
<set>  = {i+5 for i in range(10)}                   # Or: {5, 6, ..., 14}
<dict> = {i: i*2 for i in range(10)}                # Or: {0: 0, 1: 2, ..., 9: 18}
```

Range

In [ ]:
```
<range> = range(stop)                   # range(to_exclusive)
<range> = range(start, stop)            # range(from_inclusive, to_exclusive)
<range> = range(start, stop, ±step)     # range(from_inclusive, to_exclusive, ±step_size)
>>> [i for i in range(3)]
[0, 1, 2]
```

String

In [ ]:
```
<str>  = <str>.strip()                          # Strips all whitespace characters from both ends.
<str>  = <str>.strip('<chars>')                 # Strips all passed characters from both ends.
<list> = <str>.split()                          # Splits on one or more whitespace characters.
<list> = <str>.split(sep=None, maxsplit=-1)     # Splits on 'sep' str at most 'maxsplit' times.
<list> = <str>.splitlines(keepends=False)       # On [\n\r\f\v\x1c-\x1e\x85\u2028\u2029] and \r\n.
<str>  = <str>.join(<coll_of_strings>)          # Joins elements using string as a separator.
<bool> = <sub_str> in <str>                     # Checks if string contains the substring.
<bool> = <str>.startswith(<sub_str>)            # Pass tuple of strings for multiple options.
<bool> = <str>.endswith(<sub_str>)              # Pass tuple of strings for multiple options.
<int>  = <str>.find(<sub_str>)                  # Returns start index of the first match or -1.
<int>  = <str>.index(<sub_str>)                 # Same, but raises ValueError if missing.
<str>  = <str>.replace(old, new [, count])      # Replaces 'old' with 'new' at most 'count' times.
<str>  = <str>.translate(<table>)               # Use `str.maketrans(<dict>)` to generate table.
<str>  = chr(<int>)                             # Converts int to Unicode character.
<int>  = ord(<str>)                             # Converts Unicode character to int.
```

In [ ]:
```
+---------------+----------+----------+----------+----------+----------+
|               | [ !#$%…] | [a-zA-Z] |  [¼½¾]   |  [²³¹]   |  [0-9]   |
+---------------+----------+----------+----------+----------+----------+
| isprintable() |   yes    |   yes    |   yes    |   yes    |   yes    |
| isalnum()     |          |   yes    |   yes    |   yes    |   yes    |
| isnumeric()   |          |          |   yes    |   yes    |   yes    |
| isdigit()     |          |          |          |   yes    |   yes    |
| isdecimal()   |          |          |          |          |   yes    |
+---------------+----------+----------+----------+----------+----------+
```

Regex

In [ ]:
```
import re
<str>   = re.sub(<regex>, new, text, count=0)  # Substitutes all occurrences with 'new'.
<list>  = re.findall(<regex>, text)            # Returns all occurrences as strings.
<list>  = re.split(<regex>, text, maxsplit=0)  # Use brackets in regex to include the matches.
<Match> = re.search(<regex>, text)             # Searches for first occurrence of the pattern.
<Match> = re.match(<regex>, text)              # Searches only at the beginning of the text.
<iter>  = re.finditer(<regex>, text)           # Returns all occurrences as Match objects.
```

Format

In [ ]:
```
<str> = f'{<el_1>}, {<el_2>}'            # Curly brackets can also contain expressions.
<str> = '{}, {}'.format(<el_1>, <el_2>)  # Or: '{0}, {a}'.format(<el_1>, a=<el_2>)
<str> = '%s, %s' % (<el_1>, <el_2>)      # Redundant and inferior C-style formatting.
```

Funciones Basicas

In [ ]:
```
<num> = pow(<num>, <num>)                    # Or: <num> ** <num>
<num> = abs(<num>)                           # <float> = abs(<complex>)
<num> = round(<num> [, ±ndigits])            # `round(126, -1) == 130`
```

Math

In [ ]:
```
from math import e, pi, inf, nan, isinf, isnan    # `<el> == nan` is always False.
from math import sin, cos, tan, asin, acos, atan  # Also: degrees, radians.
from math import log, log10, log2                 # Log can accept base as second arg.
```

Statistics

In [ ]:
```
from statistics import mean, median, variance     # Also: stdev, quantiles, groupby.
```

Random

In [ ]:
```
from random import random, randint, choice        # Also: shuffle, gauss, triangular, seed.
<float> = random()                                # A float inside [0, 1).
<int>   = randint(from_inc, to_inc)               # An int inside [from_inc, to_inc].
<el>    = choice(<sequence>)                      # Keeps the sequence intact.
```

Datetime

- El módulo 'datetime' proporciona las clases 'date' , 'time' , 'datetime'
  y 'timedelta' Todos son inmutables y hashable.
- Los objetos de hora y fecha y hora pueden ser 'conscientes' , lo que significa que tienen una zona horaria definida, o 'naive' , lo que significa que no la tienen.
- Si el objeto es ingenuo, se supone que se encuentra en la zona horaria del sistema.

In [ ]:
```
from datetime import date, time, datetime, timedelta
from dateutil.tz import UTC, tzlocal, gettz, datetime_exists, resolve_imaginary
```

Constructores

In [ ]:
```
<D>  = date(year, month, day)                # Only accepts valid dates from 1 to 9999 AD.
<T>  = time(hour=0, minute=0, second=0)      # Also: `microsecond=0, tzinfo=None, fold=0`.
<DT> = datetime(year, month, day, hour=0)    # Also: `minute=0, second=0, microsecond=0, …`.
<TD> = timedelta(weeks=0, days=0, hours=0)   # Also: `minutes=0, seconds=0, microseconds=0`.
```

Now

In [ ]:
```
<D/DTn>  = D/DT.today()                  # Current local date or naive datetime.
<DTn>    = DT.utcnow()                   # Naive datetime from current UTC time.
<DTa>    = DT.now(<tzinfo>)              # Aware datetime from current tz time.
```

Timezone

In [ ]:
```
<tzinfo> = UTC                           # UTC timezone. London without DST.
<tzinfo> = tzlocal()                     # Local timezone. Also gettz().
<tzinfo> = gettz('<Continent>/<City>')   # 'Continent/City_Name' timezone or None.
<DTa>    = <DT>.astimezone(<tzinfo>)     # Datetime, converted to the passed timezone.
<Ta/DTa> = <T/DT>.replace(tzinfo=<tzinfo>)  # Unconverted object with a new timezone.
```

Format

In [ ]:
```
>>> dt = datetime.strptime('2015-05-14 23:39:00.00 +0200', '%Y-%m-%d %H:%M:%S.%f %z')
>>> dt.strftime("%A, %dth of %B '%y, %I:%M%p %Z")
"Thursday, 14th of May '15, 11:39PM UTC+02:00"
```

Funciones

In [ ]:
```
def func(<nondefault_args>): ...                # def func(x, y): ...
def func(<default_args>): ...                   # def func(x=0, y=0): ...
def func(<nondefault_args>, <default_args>): ...  # def func(x, y=0): ...
def f(*args): ...               # f(1, 2, 3)
def f(x, *args): ...            # f(1, 2, 3)
def f(*args, z): ...            # f(1, 2, z=3)

def get_multiplier(a):
    def out(b):
        return a * b
    return out
```

Lambda

In [ ]:
```
<func> = lambda: <return_value>                  # A single statement function.
<func> = lambda <arg_1>, <arg_2>: <return_value>   # Also accepts default arguments.
```

Imports

- El paquete es una colección de módulos, pero también puede definir sus propios objetos.
- En un sistema de archivos, esto corresponde a un directorio de archivos de Python con un script de inicio opcional.
- La ejecución 'import 'no brinda acceso automático a los módulos del paquete a menos que se importen explícitamente en su secuencia de comandos de inicio.

In [ ]:
```
import <module>           # Imports a built-in or '<module>.py'.
import <package>          # Imports a built-in or '<package>/__init__.py'.
import <package>.<module> # Imports a built-in or '<package>/<module>.py'.
```